Question Bank — Week 6

Stacks (Lecture 06) — Questions

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice **1**

Part B — Short answer and essay **7**

Part C — Trace the code and the algorithms **8**

Part D — Find and fix the bug **8**

Part E — Write the code **9**

> ***Answers are in a separate file:*** *`week06-answers.md`. Levels:* ***[what]*** *recall ·* ***[how]*** *apply ·* ***[why]*** *explain and justify. `Stack` is the class of `dsa/stack.py`: `push`, `pop`, `peek`, `is_empty`, `len`, with `pop` and `peek` raising `IndexError` on an empty stack. Operators in expressions are `+ - * /` with the usual precedence, all left associative.*

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W6-M01 – W6-M40
B	Short answer and essay	W6-E1 – W6-E9
C	Trace the code and the algorithms	W6-T1 – W6-T5
D	Find and fix the bug	W6-B1 – W6-B4
E	Write the code — checked by `pytest`	W6-C1 – W6-C5

Part A — Multiple choice

W6-M01 [what] "LIFO" means:

- **a)** the last item pushed is the first one popped
- **b)** the first item pushed is the first one popped
- **c)** items are popped in sorted order
- **d)** any item can be popped

W6-M02 [how] `push(1)`, `push(2)`, `push(3)`, `pop()`, `push(4)`, `pop()`. What does the **next** `pop()` return?

- **a)** 4
- **b)** 3
- **c)** 2
- **d)** 1

W6-M03 [why] A stack built on a dynamic array should keep its top at:

- **a)** index 0
- **b)** the end — the last used slot
- **c)** the middle
- **d)** it makes no difference

W6-M04 [why] A stack built on a singly linked list should keep its top at:

- **a)** the last node
- **b)** the middle node
- **c)** a separate array
- **d)** the head

W6-M05 [what] `Stack().pop()` on an empty stack:

- **a)** raises `IndexError`
- **b)** returns `None`
- **c)** returns 0
- **d)** blocks until something is pushed

W6-M06 [why] A stack on an array with its top at **index 0**. `push` costs:

- **a)** $O(1)$
- **b)** amortised $O(1)$
- **c)** $O(n)$ — every element shifts
- **d)** $O(\log n)$

W6-M07 [how] Why is `"([)]"` not balanced?

- **a)** It has an odd number of brackets
- **b)** `)` arrives when the most recent opener is `[`
- **c)** The stack is not empty at the end
- **d)** A closer arrives when the stack is empty

W6-M08 [how] Which check makes `is_balanced(")")` return `False`?

- **a)** the partner check
- **b)** the empty-at-the-end check
- **c)** a length check
- **d)** a closer arrives when the stack is empty

W6-M09 [how] The postfix form of `3 + 4 * 2` is:

- **a)** `3 4 2 * +`
- **b)** `3 4 + 2 *`
- **c)** `+ 3 * 4 2`
- **d)** `3 + 4 2 *`

W6-M10 [how] The postfix form of `( 3 + 4 ) * 2` is:

- **a)** `3 4 2 * +`
- **b)** `3 + 4 * 2`
- **c)** `3 4 + 2 *`
- **d)** `* + 3 4 2`

W6-M11 [how] Evaluating the postfix `8 3 -` gives:

- **a)** −5
- **b)** 5
- **c)** 11
- **d)** an error

W6-M12 [how] Evaluating the postfix `2 3 4 * +` gives:

- **a)** 20
- **b)** 9
- **c)** 24
- **d)** 14

W6-M13 [what] In the shunting-yard algorithm, when an operator arrives, it first pops to the output every stacked operator with:

- **a)** higher or equal precedence
- **b)** lower precedence
- **c)** any precedence
- **d)** strictly higher precedence only

W6-M14 [why] Why does an operator of **equal** precedence also leave the stack?

- **a)** It is faster
- **b)** To keep the stack small
- **c)** So that `-` and `/` are left associative: `8 - 3 - 2` is `(8 - 3) - 2`
- **d)** It makes no difference to the result

W6-M15 [how] The postfix form of `8 - 3 - 2` is:

- **a)** `8 3 2 - -`
- **b)** `8 3 - 2 -`
- **c)** `- - 8 3 2`
- **d)** `8 - 3 2 -`

W6-M16 [how] The worst-case extra space of `is_balanced` on a text of length n is:

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n^2)$
- **d)** $O(n)$

W6-M17 [why] To turn a recursive algorithm into a loop, in general you need:

- **a)** an explicit stack
- **b)** a queue

- **c)** a sorted array
- **d)** nothing — every recursion is already a loop

W6-M18 [what] In an undo/redo system with two stacks, what happens to the redo stack when the user performs a **new** action?

- **a)** the new action is pushed on it
- **b)** it is cleared
- **c)** its top is undone
- **d)** nothing

W6-M19 [how] n pushes onto the `Stack` of `dsa/stack.py` (built on a `DynamicArray`) cost in total:

- **a)** $O(n^2)$
- **b)** $O(n \log n)$
- **c)** $O(n)$ — amortised $O(1)$ each
- **d)** $O(1)$

W6-M20 [what] Which of these is **not** a natural stack application?

- **a)** checking balanced brackets
- **b)** evaluating postfix expressions
- **c)** undo in an editor
- **d)** serving customers in the order they arrived

W6-M21 [how] While `is_balanced("(((())))")` runs, the largest number of items on the stack at once is:

- **a)** 4
- **b)** 8
- **c)** 1
- **d)** 2

W6-M22 [what] `peek()` differs from `pop()` in that it:

- **a)** is slower
- **b)** returns the top without removing it
- **c)** returns the bottom
- **d)** never raises an error

W6-M23 [how] The shunting-yard algorithm converts `2 * 3 + 4` to:

- **a)** `2 3 4 + *`
- **b)** `2 3 * 4 +`
- **c)** `2 * 3 4 +`
- **d)** `+ * 2 3 4`

W6-M24 [how] The shunting-yard algorithm converts `( 1 + 2 ) * ( 3 - 4 ) / 5` to:

- **a)** `1 2 + 3 4 - 5 / *`
- **b)** `1 2 3 4 + - * 5 /`
- **c)** `1 2 + 3 4 - * 5 /`

- **d)** `1 2 + * 3 4 - 5 /`

W6-M25 [why] A student's `infix_to_postfix` pops an operator only when the stacked one has **strictly higher** precedence (> instead of >=). Their calculator, `evaluate_postfix(infix_to_postfix(...))` now gives `8 - 3 - 2` as:

- **a)** 3
- **b)** 13
- **c)** a `ValueError`
- **d)** 7

W6-M26 [why] With the same > bug, which of these inputs is **still** converted correctly?

- **a)** `3 + 4 * 2`
- **b)** `8 - 3 - 2`
- **c)** `8 / 4 / 2`
- **d)** `6 - 2 + 1`

W6-M27 [how] `evaluate_postfix(["6", "2", "/", "3", "*"])` returns:

- **a)** 1.0
- **b)** 18.0
- **c)** 9.0
- **d)** it raises `ValueError`

W6-M28 [why] A buggy `evaluate_postfix` computes (first value popped) op (second value popped): the operands the wrong way round. On which input does it **still** give the right answer?

- **a)** `8 3 -`
- **b)** `9 3 /`
- **c)** `5 1 2 + -`
- **d)** `3 4 + 2 *`

W6-M29 [how] `evaluate_postfix(["3", "4"])`:

- **a)** returns 4
- **b)** raises `ValueError`: two values are left at the end
- **c)** returns 7
- **d)** returns 3

W6-M30 [what] `evaluate_postfix(["1", "0", "/"])`:

- **a)** raises `ZeroDivisionError`
- **b)** raises `ValueError`
- **c)** returns 0
- **d)** returns `None`

W6-M31 [why] A version of `is_balanced` ends with `return True` instead of `return openers.is_empty()`. Which input does it get **wrong**?

- **a)** `([)]`
- **b)** `)(`
- **c)** `(a))`

- **d)** `((a)`

W6-M32 [what] A version of `is_balanced` pops for every closer **without** first checking that the stack is non-empty. On `"())"` it:

- **a)** returns `True`
- **b)** returns `False`
- **c)** raises `IndexError`: pop on the empty stack
- **d)** loops for ever

W6-M33 [how] `is_balanced("a(b)c)d(")` returns `False`. Which check fails **first**?

- **a)** the partner check
- **b)** a closer arrives when the stack is empty
- **c)** the empty-at-the-end check
- **d)** none of them: it returns `True`

W6-M34 [why] `is_balanced("print('(')")`, a line of valid Python, returns:

- **a)** `True`: the characters inside quotes are skipped
- **b)** `True`: it has as many `(` as `)`
- **c)** it raises an error on the quote characters
- **d)** `False`: the `(` inside the quotes is counted like any other

W6-M35 [why] Most pushes onto the `Stack` of `dsa/stack.py` are instant, but now and then **one** push takes much longer. Why?

- **a)** The `DynamicArray` was full, so that push copies every item into a bigger block: $O(n)$ for that push, amortised $O(1)$ overall
- **b)** The top is kept at index 0, so that push shifts every item
- **c)** `push` walks to the end of the stack first
- **d)** Python checks the whole stack for duplicates

W6-M36 [what] Which stack gives **worst-case** $O(1)$ for every push, not just amortised $O(1)$?

- **a)** an array with the top at the end
- **b)** an array with the top at index 0
- **c)** a singly linked list with the top at the head
- **d)** a singly linked list with the top at the last node

W6-M37 [what] `s = Stack([1, 2, 3])`. What does `s.pop()` return?

- **a)** 1
- **b)** 3
- **c)** `[1, 2, 3]`
- **d)** it raises `IndexError`

W6-M38 [why] Two `peek()` calls in a row, with nothing in between, return two **different** values. The most likely bug is that `peek`:

- **a)** reads index 0 instead of the last slot
- **b)** forgets to raise `IndexError` on an empty stack
- **c)** is $O(n)$ instead of $O(1)$

- **d)** removes the top, as pop does

W6-M39 [how] An editor with an undo stack and a redo stack. The user types A, types B, types C, then: undo, undo, redo, types D. What is on the two stacks (bottom → top)?

- **a)** undo: A B D · redo: empty
- **b)** undo: A B C D · redo: empty
- **c)** undo: A B D · redo: C
- **d)** undo: A D · redo: C B

W6-M40 [why] `nested_sum` with your own `Stack` instead of recursion. Its real advantage in Python is that:

- **a)** it is asymptotically faster
- **b)** it adds the numbers in a different order, so the sum is more exact
- **c)** a deeply nested input cannot raise `RecursionError`: your `Stack` has no 1,000-frame limit
- **d)** it uses no extra memory

Part B — Short answer and essay

W6-E1 [why] *(4 marks)* State the Stack ADT with the cost of each operation. Describe two honest implementations, compare them, and describe one design that passes every test but is not honest.

W6-E2 [how] *(4 marks)* Describe the stack algorithm for balanced brackets. Give an example of each of the three ways a text can be unbalanced, and the check that catches it.

W6-E3 [how] *(3 marks)* Explain how a stack evaluates a postfix expression. Why does the order in which the two operands are popped matter, and for which operators?

W6-E4 [why] *(4 marks)* Explain the shunting-yard algorithm, and trace it on `2 * ( 3 + 4 ) - 5`. Why must an operator of equal precedence be popped?

W6-E5 [why] *(3 marks)* "Recursion is a stack." Explain, and describe how you would rewrite a recursive algorithm without recursion. When is that worth doing in Python?

W6-E6 [why] *(4 marks)* A student's `infix_to_postfix` pops an operator only on **strictly higher** precedence (> instead of >=). Give one input that it still converts correctly and one that it gets wrong, with both postfix results and their values. Explain why the tests can miss this bug, and name an operator for which > is the right rule.

W6-E7 [how] *(3 marks)* Run the whole calculator on `( 8 - 2 ) / ( 1 + 2 ) * 4`: convert it to postfix with the shunting-yard algorithm, then evaluate the postfix with a stack, showing the stack after each operator. Why is the answer a float?

W6-E8 [why] *(4 marks)* Compare a stack on a dynamic array (top at the end) with a stack on a singly linked list (top at the head): the cost of push in the worst case and amortised, the memory per item, and memory locality. The lecture's timings found the linked stack faster in this course. Why, and would that still hold for Python's own `list`? Which would you choose for a program that must never pause?

W6-E9 [how] *(3 marks)* Design undo and redo for a text editor with two stacks. Say what each of *action*, *Undo* and *Redo* does to the two stacks, and show both stacks after: type A, type B, type C, undo, undo, redo, type D. What would go wrong if a new action did not clear the redo stack?

Part C — Trace the code and the algorithms

W6-T1 [how] What is printed?

```
s = Stack()
out = []
s.push(1); s.push(2)
out.append(s.pop())
s.push(3); s.push(4)
out.append(s.peek())
out.append(s.pop())
out.append(s.pop())
out.append(len(s))
print(out)
```

W6-T2 [how] Evaluate the postfix expression `5 1 2 + 4 * + 3 -`, showing the stack after every token.

W6-T3 [how] Convert `2 * ( 3 + 4 ) - 5` to postfix with the shunting-yard algorithm, showing the output and the operator stack after every token.

W6-T4 [how] Run `is_balanced("{[()()]}")` by hand. Show the stack after every character. What is the maximum stack size?

W6-T5 [why] In what order are the numbers printed? Why that order?

```
pending = Stack([[1, [2, 3], 4]])
while not pending.is_empty():
    item = pending.pop()
    if isinstance(item, list):
        for inner in item:
            pending.push(inner)
    else:
        print(item, end=" ")
```

Part D — Find and fix the bug

W6-B1 [how]

```
def pop(self):
    if len(self._items) == 0:
```

```
        return None
    return self._items.pop()
```

W6-B2 [how]

```
def is_balanced(text):
    pairs = {")": "(", "]": "[", "}": "{"}
    s = Stack()
    for ch in text:
        if ch in "([{":
            s.push(ch)
        elif ch in pairs:
            if s.is_empty() or s.pop() != pairs[ch]:
                return False
    return True
```

W6-B3 [how] Part of `evaluate_postfix`:

```
if token in "+-*/":
    a = stack.pop()
    b = stack.pop()
    stack.push(apply(token, a, b))      # apply(op, left, right)
```

W6-B4 [why] Part of `infix_to_postfix`:

```
while not ops.is_empty() and ops.peek() in prec and prec[ops.peek()] > prec[token]:
    output.append(ops.pop())
ops.push(token)
```

Part E — Write the code

In `practice/week06.py`; check with `pytest tests/test_practice_week06.py -v`. Use your `Stack` as working storage.

W6-C1 [how] `reverse_words(sentence)`.

W6-C2 [why] `MinStack` — `push`, `pop`, `peek` and `get_min`, all $O(1)$. Why is keeping a single "current minimum" variable not enough?

W6-C3 [why] `next_greater(values)` in $O(n)$. Why is the total work linear, even though there is a loop inside a loop?

W6-C4 [how] `tags_balanced(html)` — the brackets algorithm, for HTML tags.

W6-C5 [how] `decode(encoded)` — expand `k[text]`, which may nest.